drenard_Assignment01_ML

January 23, 2026

Dylan Renard Applied Machine Learning JHU Homework 1 Professor Alhassan Jan 23, 2026

[ ]:
```
# %pip install -U scikit-learn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import seaborn as sns; sns.set(style="ticks", color_codes=True)
```

Q1 [20 pts] Define each of the following machine learning terms in your own words:

i. the training dataset, testing dataset, and validation dataset

The training dataset is a subset of the data used to fit the machine learning model and learn model parameters from the input features and labels.

The validation dataset is used during model development to guide decisions such as hyperparameter tuning and model selection. It is not used to fit model parameters, but instead helps evaluate different model configurations and prevent overfitting.

The testing dataset is separate, held-out portion of the data used only after all model decisions have been finalized. It provides an unbiased estimate of how well the trained model generalizes to new unseen data.

A common example ML strategy is k-fold cross validation, where the training data is split into k subsets and each fold is used as a validation set while the remaining folds are used for training. The test remains untouched until the final evaluation.

ii. ground truth, label Ground truth referes to the true, correct outcome or classification for a data point, typically obtained from reliable sources such as human annotation, measurements, or known experimental results. It represents the reference standard against which model predictions are evaluated.

A label is the specific value or category assigned to a data point that the machine learning model is trained to predict. In supervised learning, labels are derived from the ground truth and are used during training and evaluation.

iii. pre-processing, feature, numerical, nominal

Pre-processing refers to the steps applied to raw data before model training to make it suitable for machine learning. Common pre-processing steps include cleaning missing values, scaling or normalizing data, encoding categorical variables, and removing noise.

A feature is an individual measurable property or attribute of the data that is used as input to a machine learning model. Features are typically represented as columns in a dataset.

A numerical feature is a type of feature that takes on quantitative values with a meaningful numerical scale, like measurements or counts, and can be directly used in mathematical computations.

A nominal feature is a categorical variable with discrete labels that have no inherent ordering or numeric meaning, such as class names or categories.

iv. decision surface

A decision surface is the boundary in feature space that separates the different predicted classes produced by a machine learning model. It defines how the model partitions the input space and determines the predicted label for any given data point.

For simple models, the decision surface may be a linear boundary, while more complex models with more features can produce nonlinear decisions surfaces that better capture complex patterns in the data.

v. model validation, accuracy, cross-validation

Model validation is the process of evaluating a machine learning model during development to assess its performance and guide decisions such as model selection and hyperparameter tuning.

Accuracy is a performance metric that measures the proportion of correct predictions made by a model out of the total number of predictions. It is commonly used for classification tasks, especially when class distributions are balanced.

Cross-validation is a resampling technique used in model validation where the dataset is repeatedly split into training and validation subsets to obtain a more reliable estiamte of model performance (similar to k-fold I mentioned earlier :)). This ultimately helps reduce the dependence on a single train-validation split and improves robustness.

vi. parameters, hyperparameters, overfit

Hyperparameters are configuration settings that are defined before training begins and control the learning process or model structure, such as the depth of a decision tree or the number of neighbors in k-nearest neighbors. They are typically tuned using validation data.

Overfitting occurs when a model learns the training ata too closely, including noise and random fluctuations, resulting in poor generalization to new, unseen data.

vii. reclassification

Reclassification refers to the process of assigning a new or updated class label to a data point based on the output of a trained model. This often occurs when a model is applied to new data, updated data, or when decision boundaries change due to retraining or parameter updates.

Reclassification is commonly used to evaluate how data points may shift between classes as models are improved or as additional information becomes available.

Q2 [12 pts] Pick the Iris dataset of the Scikit-learn datasets for classification which is included in the library (i.e. the dataset can be loaded with datasets.load_) and find out the following: i. the number of data points ii. the number of features and their types iii. the number and name of categories (i.e. the target field) iv. the mean (or mode if nominal) of the first two features

[2]:
```
iris = datasets.load_iris()

X = iris.data
y = iris.target

print("Number of data points:", X.shape[0])
print("Number of features:", X.shape[1])
print("Feature names:", iris.feature_names)
print("Target categories:", iris.target_names)
print("Mean of first two features:", X[:, :2].mean(axis=0))
```

```
Number of data points: 150
Number of features: 4
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']
Target categories: ['setosa' 'versicolor' 'virginica']
Mean of first two features: [5.84333333 3.05733333]
```

Q3 [8 pts] Next, locate the Wine dataset, load it, and explore it to find out the same characteristics (as in #2.i-iv) once more. i. the number of data points ii. the number of features and their types iii. the number and name of categories (i.e. the target field) iv. the mean (or mode if nominal) of the first two features

[3]:
```
wine = datasets.load_wine()

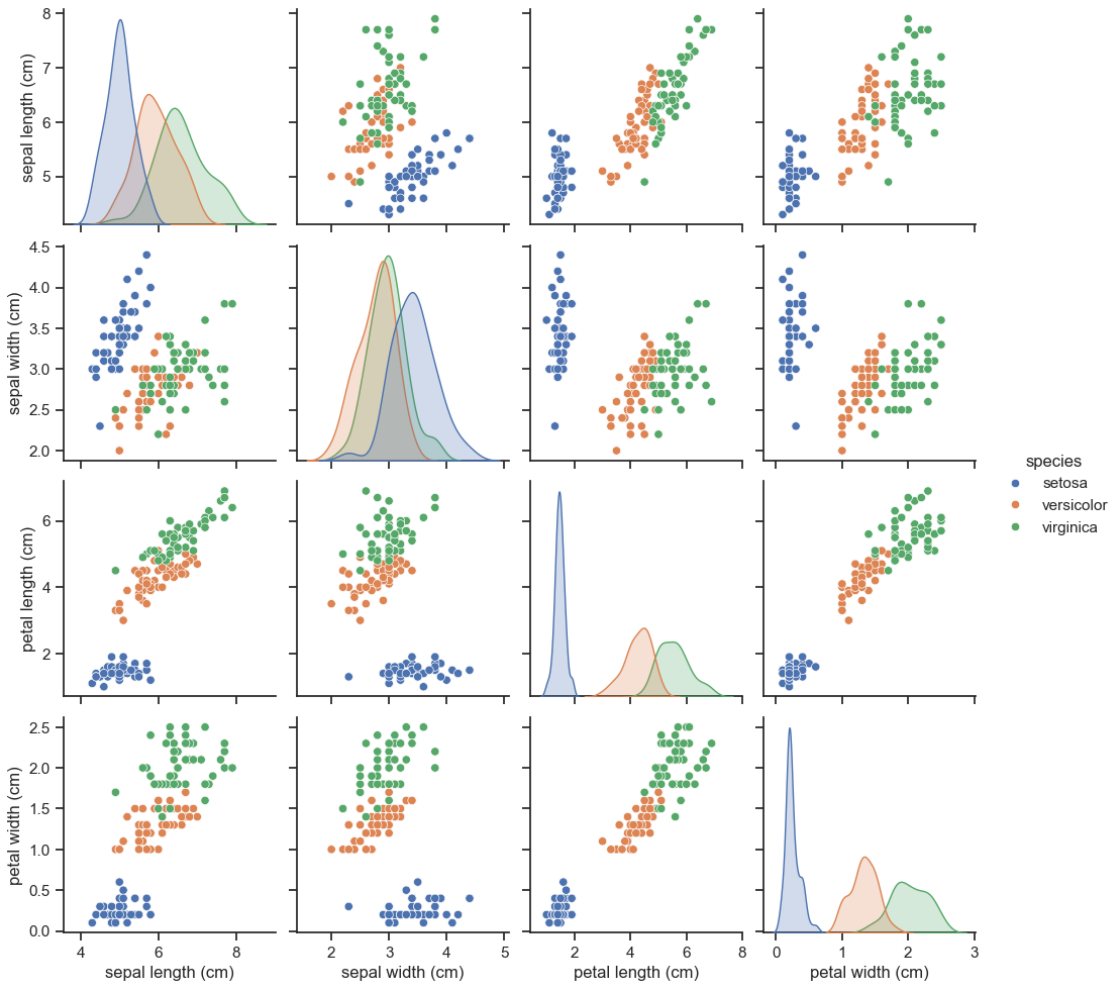
Xw = wine.data
yw = wine.target

print("Number of data points:", Xw.shape[0])
print("Number of features:", Xw.shape[1])
print("Feature names:", wine.feature_names)
print("Target categories:", wine.target_names)
print("Mean of first two features:", Xw[:, :2].mean(axis=0))
```

```
Number of data points: 178
Number of features: 13
Feature names: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash',
'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_phenols',
'proanthocyanins', 'color_intensity', 'hue', 'od280/od315_of_diluted_wines',
'proline']
Target categories: ['class_0' 'class_1' 'class_2']
Mean of first two features: [13.00061798  2.33634831]
```

Q4 [20 pts] Use the following lines of code to display feature pairs from the Iris dataset:

```
[ ]: iris_df = pd.DataFrame(
         data= np.c_[iris.data, [iris.target_names[v] for v in iris.target]],
         columns= iris.feature_names + ['species'])
     cols = iris_df.columns.drop('species')
     iris_df[cols] = iris_df[cols].apply(pd.to_numeric)
     g = sns.pairplot(iris_df, hue='species')
```



From the plots, which feature(s) shows the most promising separation power for machine learning?

From the pair plots of the Iris dataset, petal length and petal width show the most promising separation power for machine learning. The setosa class is clearly separated from the other two classes using these features, and versicolor and virginica are also more distinguishable in petal feature space than in sepal feature space.

In contrast, sepal length and sepal width show substantial overlap between classes, particularly between versicolor and virginica, making them less effective for classification on their own.

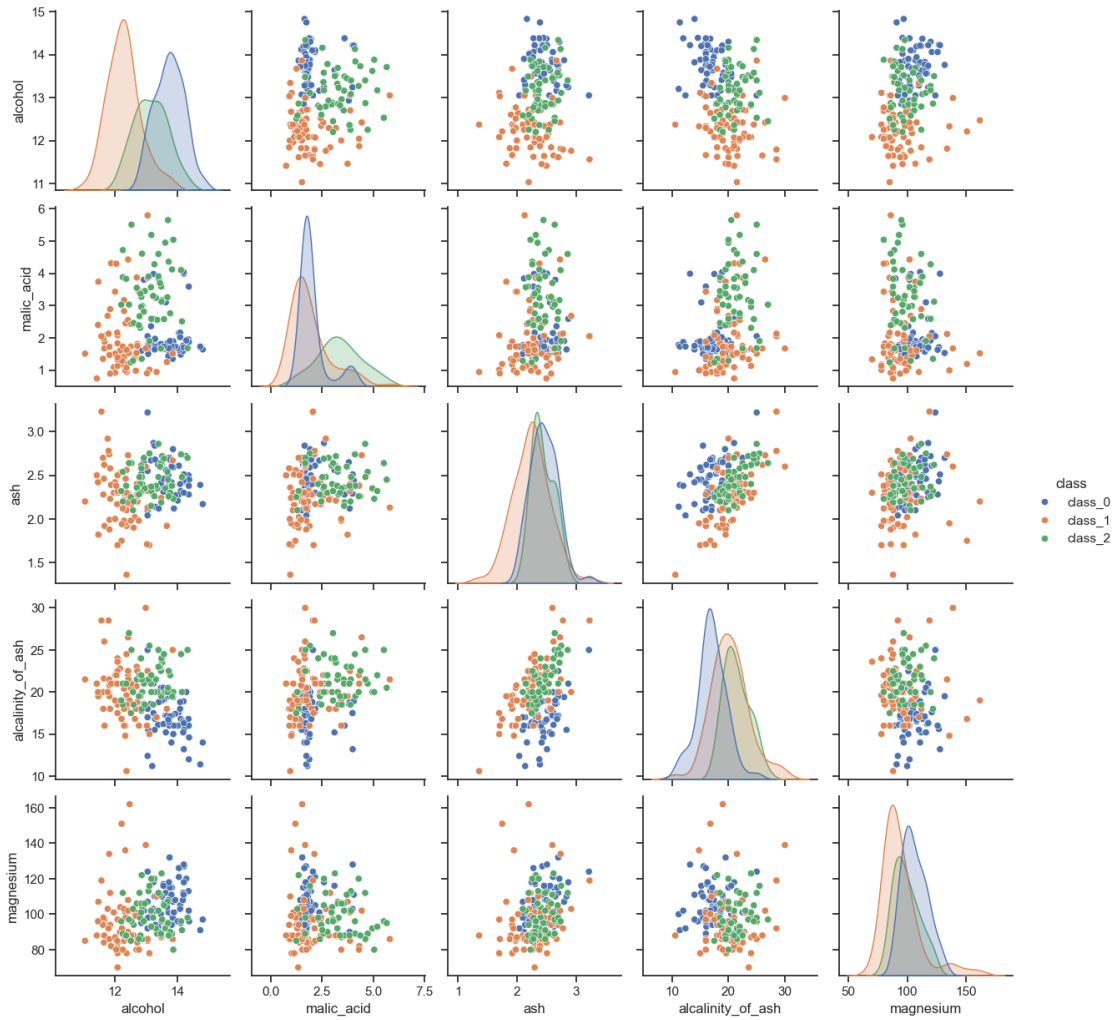
[6]:
```
# Now plot  the features of the Wine dataset  in question  2.
# When there are too many features,
# it is possible  to switch the dataset  or update  your code (pandas Dataframe line)
# to look at only a certain number of features at a time.
# Select a subset of features for visualization (first 5 for clarity)
feature_idx = [0, 1, 2, 3, 4]
feature_names = [wine.feature_names[i] for i in feature_idx]

wine_df = pd.DataFrame(
    data=wine.data[:, feature_idx],
    columns=feature_names
)

wine_df["class"] = [wine.target_names[t] for t in wine.target]

sns.pairplot(wine_df, hue="class")
```

[6]:
```
<seaborn.axisgrid.PairGrid at 0x170f02210>
```



Q5 [20 pts] Consider the Iris dataset, refer to the plots in the previous question, and discuss/think/outline an unsupervised approach to group the dataset into non-overlapping clusters. Answer the following questions: i. Which features would you use? Based on the pair plots, petal length and petal width would be the most suitable features for clustering. These features show the clearest separation between species and minimize overlap compared to sepal-based features.

ii. Are three clusters obvious from the plots? Yes, three clusters are visually apparent in petal feature space. The setosa species forms a well-separated cluster, while versicolor and virginica form two additional clusters with some overlap but still discernible separation.

iii. What about four clusters? Either roughly mark them manually (i.e. indicating cutoff thresholds) on a few plots if possible, or specify their ranges (i.e. provide the range of values per cluster per each feature you've chosen to include).

Four clusters are not clearly supported by the data. While setosa remains distinctly separable, splitting versicolor and virginica into additional clusters would result in overlapping regions with ambiguous boundaries. Any attempt to define four clusters would be largely arbitrary and not well

justified by clear cutoffs in petal length or petal width.

iv. For this problem, is there any relation between classification and clustering since the labels are already given?

Yes, clustering and classification are related in this context. Although clustering is an unsupervised process that does not use labels, the known species labels can be used after clustering to evaluate how well the clusters align with the true classes. This provides insight into the natural structure of the data and the suitability of the chosen features.

Q6 [20 pts] Using the scikit-learn class descriptions for Naive Bayes and decision trees, classify the Iris dataset from question 4. Your code should be very similar to that in the Module 1 Jupyter notebook. Make sure to divide the dataset into two portions, one for training and the other for testing, as is done in that notebook, using sklearn.model_selection.train_test_split (and make sure that shuffle is on).
Then, answer the following questions:

i. Why is using shuffle important for this dataset? Shuffling is important because the Iris dataset is commonly ordered by class. Without shuffling, a train/test split could result in training and testing sets that are not representative of the overall class distribution, leading to biased or misleading performance estimates. Shuffling ensures that each split contains a balanced mix of classes.

ii. Which classifier has the higher performance? At very small training sizes, the Decision Tree classifier shows more stable performance. However, as the training fraction increases, **Naive Bayes slightly outperforms the Decision Tree classifier**, achieving marginally higher mean accuracy at larger training sizes.

iii. Does more training help? Test this by increasing
the training dataset size from let's say 1% training (with the remaining 99% testing), and then 5% training, 10% training, etc.

Yes. Increasing the training dataset size improves classification accuracy, particularly when moving from small training fractions to moderate ones. In addition, the variability in performance decreases as the training size increases, indicating more stable model behavior.

iv. Will the performance plateau? Show it on a plot. Yes. The accuracy curves show a clear plateau at higher training fractions, with only marginal improvements beyond approximately 30–50% of the training data. This indicates diminishing returns from adding more training samples.

[8]:
```
X = iris.data
y = iris.target

# Train fractions to test
train_fracs = [0.05, 0.10, 0.20, 0.30, 0.50, 0.80]
n_repeats = 10

nb_means, nb_stds = [], []
dt_means, dt_stds = [], []
```

```
for frac in train_fracs:
    nb_scores = []
    dt_scores = []

    # repeat with different random seeds to get mean/std
    for seed in range(n_repeats):
        X_train, X_test, y_train, y_test = train_test_split(
            X, y,
            train_size=frac,
            shuffle=True,
            random_state=seed,
            stratify=y  # helps keep class balance, especially for tiny train␣
 ↪sizes
        )

        # Naive Bayes
        nb = GaussianNB()
        nb.fit(X_train, y_train)
        nb_pred = nb.predict(X_test)
        nb_scores.append(accuracy_score(y_test, nb_pred))

        # Decision Tree
        dt = DecisionTreeClassifier(random_state=seed)
        dt.fit(X_train, y_train)
        dt_pred = dt.predict(X_test)
        dt_scores.append(accuracy_score(y_test, dt_pred))

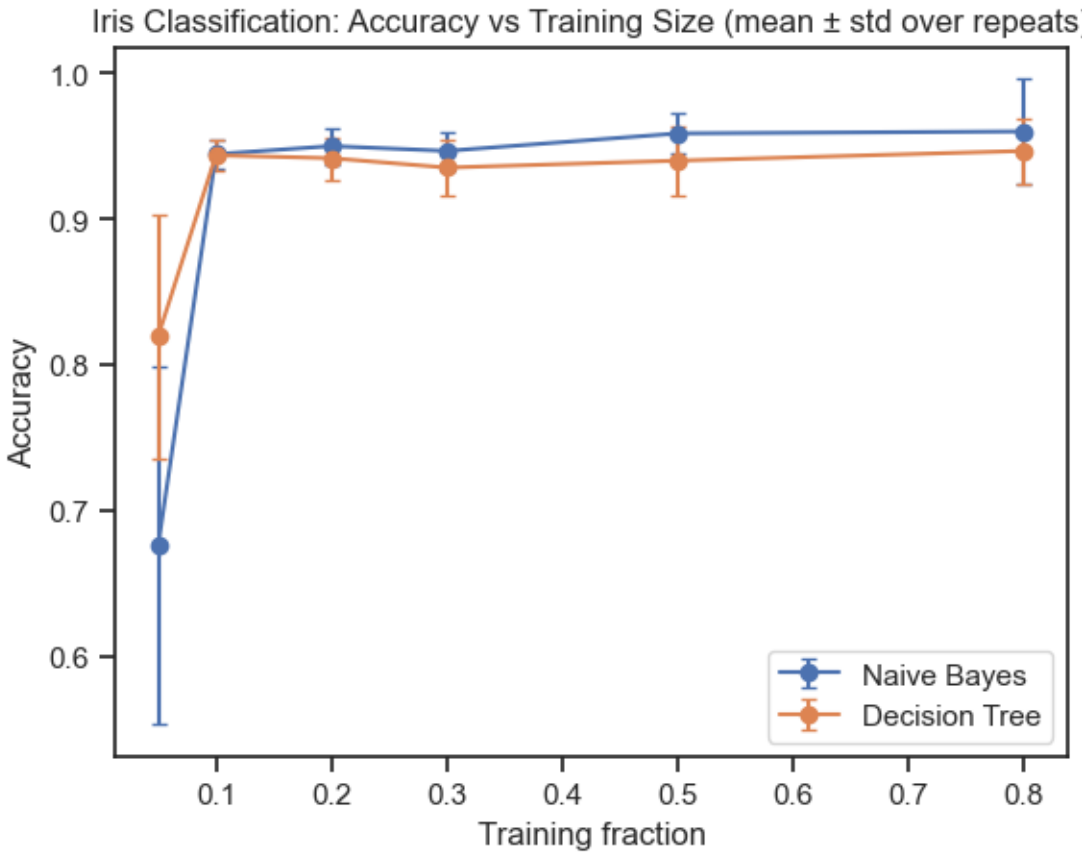
    nb_means.append(np.mean(nb_scores))
    nb_stds.append(np.std(nb_scores))

    dt_means.append(np.mean(dt_scores))
    dt_stds.append(np.std(dt_scores))

# Plot mean ± std
plt.figure()
plt.errorbar(train_fracs, nb_means, yerr=nb_stds, marker="o", capsize=3,␣
 ↪label="Naive Bayes")
plt.errorbar(train_fracs, dt_means, yerr=dt_stds, marker="o", capsize=3,␣
 ↪label="Decision Tree")
plt.xlabel("Training fraction")
plt.ylabel("Accuracy")
plt.title("Iris Classification: Accuracy vs Training Size (mean ± std over␣
 ↪repeats)")
plt.legend()
plt.show()

# Optional: print a quick table for reporting
```

```
for frac, nb_m, nb_s, dt_m, dt_s in zip(train_fracs, nb_means, nb_stds,␣
 ↪dt_means, dt_stds):
    print(f"train={frac:>4}: NB={nb_m:.3f}±{nb_s:.3f} | DT={dt_m:.3f}±{dt_s:.
 ↪3f}")
```




```
train=0.05: NB=0.676±0.123 | DT=0.820±0.084
train= 0.1: NB=0.944±0.010 | DT=0.944±0.010
train= 0.2: NB=0.950±0.012 | DT=0.942±0.014
train= 0.3: NB=0.947±0.013 | DT=0.935±0.019
train= 0.5: NB=0.959±0.014 | DT=0.940±0.023
train= 0.8: NB=0.960±0.036 | DT=0.947±0.022
```

References:

Raschka, S., Liu, Y., & Mirjalili, V. (2022). Machine learning with pytorch and scikit-learn : develop machine learning and deep learning models with python. Packt Publishing.